You are right to catch that! I summarized the first six assignments in that specific section, but there are actually **10 distinct implementations** across the files you provided (6 in Java for AI and 4 in Python for Information Security).

Here is the complete, detailed breakdown for all 10 assignments, specifically formatted to help you answer questions from an external examiner.

---

**1. BFS & DFS (Binary Tree Traversal)**

- **Node Class**: This is the basic data structure that defines the tree. It contains data (the value), and left/right pointers to its children.
- **Recursive DFS (Preorder, Inorder, Postorder)**:
    - **Logic**: Uses the **System Stack** through recursion. The function calls itself deeper into the tree until it hits null.
    - **Preorder**: Root $\rightarrow$ Left $\rightarrow$ Right (used for copying trees).
    - **Inorder**: Left $\rightarrow$ Root $\rightarrow$ Right (returns sorted data in a BST).
    - **Postorder**: Left $\rightarrow$ Right $\rightarrow$ Root (used for deleting trees).
- **BFS_levelOrder**:
    - **Logic**: Uses a Queue (FIFO). It visits nodes level-by-level, ensuring every node at the current depth is processed before moving deeper.

---

**2. A* Algorithm (8-Puzzle)**

- **calculateCost()**: This calculates the **Heuristic ($h$)** by counting the number of "misplaced tiles" relative to the final goal state.
- **solve()**: This is the heart of the algorithm. It uses a PriorityQueue to always expand the "cheapest" state first based on $f(n) = g(n) + h(n)$.
    - **$g(n)$**: The number of moves (level) from the start.
    - **$h(n)$**: The heuristic cost (misplaced tiles).
- **newNode()**: Every move creates a new object to track the board state, its parent (for path tracing), and the new blank tile (0) coordinates.

---

**3. Selection Sort**

- **Nested Loop Logic**: The outer loop moves the boundary of the sorted region, while the inner loop searches the unsorted region for the **minimum element**.
- **minIdx**: A variable used to keep track of the index where the smallest value is found during each pass.
- **Swapping**: Once the smallest element is found, it is swapped with the element at the current starting position of the unsorted region.

---

**4. N-Queens Problem (Backtracking)**

- **helper()**: A recursive function that attempts to place a queen in each column. If a placement leads to a dead end (no safe spots in the next column), it **backtracks** by removing the queen (board[i][col]=0) and trying the next row.
- **safe()**: Checks if a queen can be placed without being attacked. It only checks the row to the left and the two diagonals to the left, as queens are placed column-by-column from left to right.

---

**5. Chatbot**

- **getBotResponse()**: This uses **Keyword Matching** logic. It scans the userInput string for specific words like "fees," "courses," or "faculty" and returns a pre-defined answer.
- **while(true) Loop**: Keeps the program running so the user can ask multiple questions until they type "bye".

---

**6. Expert System (Stock Trading)**

- **Knowledge Base**: The system asks the user for specific facts (Trend, Fundamentals, Indicators).
- **Evaluate()**: This acts as the **Inference Engine**. It uses a strict Boolean rule: if all three conditions are positive, it triggers a "Buy" recommendation.

---

**7. Diffie-Hellman Key Exchange (Python)**

- **Modular Exponentiation**: Uses math.pow(g, x) % n. The security relies on the **Discrete Logarithm Problem**, making it hard to find $x$ even if you know $g, n,$ and $A$.
- **Shared Secret**: Alice and Bob arrive at the same key (k1 == k2) because $(g^x)^y \pmod n$ is mathematically identical to $(g^y)^x \pmod n$.

---

**8. RSA Algorithm (Python)**

- **Key Generation**:
  - p, q: Two prime numbers used to calculate $n$ (part of the public key).
  - m: Euler's totient $(p-1)(q-1)$.
  - e (Public Key): Coprime to $m$.
  - d (Private Key): The modular inverse of $e$.
- **encrypt/decrypt**: Uses the pow(base, exp, mod) function for efficiency and security.

---

**9. Transposition Cipher (Python)**

- **ciphertext Generation**: The plaintext is broken into "columns" based on the key. The code iterates through the string using pointer += key to pick characters for each column.
- **decryptMessage()**: Reconstructs the grid by calculating the number of rows and "shaded boxes" (empty slots in the grid) to read the characters back in the correct sequence.

---

**10. XOR and AND Operations (Python)**

- **ord(char)**: Converts a character into its integer ASCII value.
- **chr()**: Converts the modified integer back into a character for display.
- **Logic**:
  - **AND (& 127)**: A mask that keeps only the first 7 bits of a character.
  - **XOR (^ 127)**: A reversible bit-flip that obfuscates the text. If you XOR the result again with 127, you get the original "Hello, World!" back.